



FACULTY OF ACCOUNTING AND INFORMATICS

DEPARTMENT OF INFORMATION TECHNOLOGY

INTEGRATIVE PROGRAMMING AND TECHNOLOGIES III

TEST 2 (Data mapping and exchange) Memo

DATE: 7 MAY 2025

TIME: 45 minutes

EXAMINER: BOAMAH-ABU, CHARLES (PhD)

TOTAL MARKS: 35

NO. OF PAGES: 6 (Including cover page)

INSTRUCTIONS/REQUIREMENTS

1. *This examination is closed book*
2. *Answer all questions*
3. *Where stated "Briefly answer", please give a short answer*
4. *Where stated "Write the missing parts" of code, please write only the missing parts*
5. *Ensure your handwriting is readable*

Part 1

5 marks

In each of the following, state whether the statement is **true** or **false**. Write **T** (for true) or **F** (for false)

1. XML is based on the relational data structure.
2. XML represents data as primitives.
3. An XML attribute can have only one value.
4. XSL is a type of XML schema.
5. XSLT uses an XSL document to transform an XML document to a web document.

Answer: 1. F 2. F 3. T 4. F 5. T

Part 2

30 marks

NLM College has many campuses. The data on campuses is stored as XML code in a file called **campus.xml**. Below is part of the XML code. The root element is **College**. This contains many **Campus** elements. Each **Campus** has a **code** (a unique order **code**) and **num** (the **number** of students on that campus).

```
<?xml version="1.0" encoding="UTF-8"?>
<College>
  <Campus code="c1">
    <num>100</num>
  </Campus>
  <Campus code="c2">
    <num>120</num>
  </Campus>
  <Campus code="c3">
    <num>150</num>
  </Campus>
</College>
```

Use the information to answer **Questions 1, 2 and 3** below.

Question 1 [6]

1. Below is the DTD code (in a file **campus.dtd**) that defines the structure of the XML in **campus.xml**. Complete the code. (6)

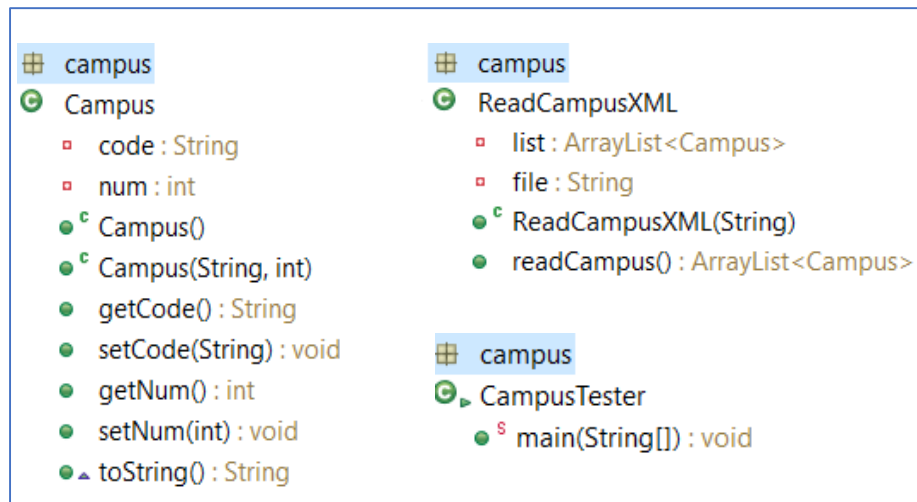
```
<?xml version="1.0" encoding="UTF-8"?>
```

Answer:

```
<?xml version="1.0" encoding="UTF-8"?>
<!ELEMENT College (Campus)*> ✓✓
<!ELEMENT Campus (num)*> ✓
<!ATTLIST Campus code CDATA #REQUIRED> ✓✓
<!ELEMENT num (#PCDATA)> ✓
```

Question 2 [16]

Below are the classes in a Java application to read the XML code from the file **campus.xml**. The class **ReadCampusXML** reads data from the file to create instances of **Campus** which are stored in a list.



Campus has two data members: **code** (code for a campus) and **num** (the number of students). The methods are: a **constructor**; **get** and **set**; and **toString**. Assume the class **Campus** has been implemented.

ReadCampusXML has one data member: **file** (i.e., the name of the XML file, a string). **list**, a static variable, is a list to store instances of **Campus**.

- The constructor initializes **file** with a file name.

- The method **readCampus** creates an instance (called **xr**) of an **XMLStreamReader** to read the **code** and **num** values from the XML file to create and store instances of **Campus** in **list**. **readCampus** returns **list** (the list of instances of **Campus**).

CampusTester creates an instance of **ReadCampusXML**, calls **readCampus** and displays the instances of **Campus** in **list**.

Part of the code for **ReadCampusXML** is shown below.

1. Give one reason why **readCampus** must handle exceptions. (2)
2. Complete the method header of **readCampus** to declare all relevant exceptions. (3)
3. In the loop in **readCampus**:
 - 3.1. Write a statement to get an event and assign to the variable **ev**. (2)
 - 3.2. Write the code to get the text in an element and assign to **val**, if the event is characters. (4)
 - 3.3. Write the code such that if an event is “end element” and the local name is “Campus”, add an instance (**c**) of **Campus** to array list **list**. (5)

```
package campus;
import java.io.*;
import java.util.*;

import javax.xml.stream.*;

public class ReadCampusXML {
    private ArrayList<Campus> list=new ArrayList<Campus>();
    private String file;
    public ReadCampusXML(String filePar) {
        file = filePar;
    }

    _____readCampus()_____ {
        FileReader fr=new FileReader(file);
        XMLInputFactory factory=XMLInputFactory.newInstance();
        XMLStreamReader xr=factory.createXMLStreamReader(fr);
        String value=null;//to store character values
        Campus c=null;//to store instance of Campus
        int ev;//to store the value of an event
        while(xr.hasNext()) {
            ev=_____;//Answer Question 3.1

            //Answer Question 3.2
            _____
            _____
            _____

            //Answer Question 3.3
            _____
            _____
            _____
        }
    }
}
```

```

    }
    fr.close();
    xr.close();

    return list;
}
}

```

Answers:

1. Any correct answer, e.g., **readCampus** creates and uses streams, which can throw exceptions. ✓✓
2. **public** ArrayList<Campus> ✓✓ readCampus() **throws** Exception ✓

3.1-3.3

```

ev=xr.next();✓✓

//Any other correct answer, e.g.:
if(ev==XMLStreamConstants.CHARACTERS) ✓✓{
    value=xr.getText();✓✓
}

if((ev==XMLStreamConstants.END_ELEMENT)&&(xr.getLocalName().equals("Campus
"))){ ✓✓✓✓
    list.add(c); ✓
}

```

Question 3**[8]**

The data in **campus.xml** is to be displayed in a web page as shown below.

c1	100
c2	120
c3	150

1. The XSL code (below) is in a file **campus.xsl** and formats the XML data in file **campus.xml**. Complete the XSL code.

(6)

```
<xsl:template match="/">
  <table>
    <xsl:for-each select="_____">
      <tr>
        <td> _____ </td>
        <td> _____ </td>
      </tr>
    </xsl:for-each>
  </table>
</xsl:template>
```

Answer:

```
<xsl:template match="/">
  <table>
    <xsl:for-each select="College/Campus"✓✓>
      <tr>
        <td><xsl:value-of select="@code"/>✓✓</td>
        <td><xsl:value-of select="num"/>✓✓</td>
      </tr>
    </xsl:for-each>
  </table>
</xsl:template>
```

2. Complete the line of code (below) in **campus.xml** to link the XML data to **campus.xsl**.

(2)

```
<?xml-stylesheet type="_____" href="_____"?>
```

Answer:

```
<?xml-stylesheet type="text/xsl" href="campus.xsl"?>
```